

Object-oriented Programming

Week 5

What are initializer lists in C++?

- The **initializer list** is used to directly initialize data members of a class. An initializer list starts after the constructor name and its parameters. The list begins with a colon (:) and is followed by the list of variables that are to be initialized – all of the variables are separated by a comma with their values in curly brackets.

Syntax

```
Constructorname(datatype value1, datatype value2):datamember(value  
1),datamember(value2)  
{  
    ...  
}
```

Example

```
#include <iostream>
class car{
    int a;
    public:
        car(int e):a(e){}
        // the above initialization is same as
        // car(int e){
        //     a=e;
        // }

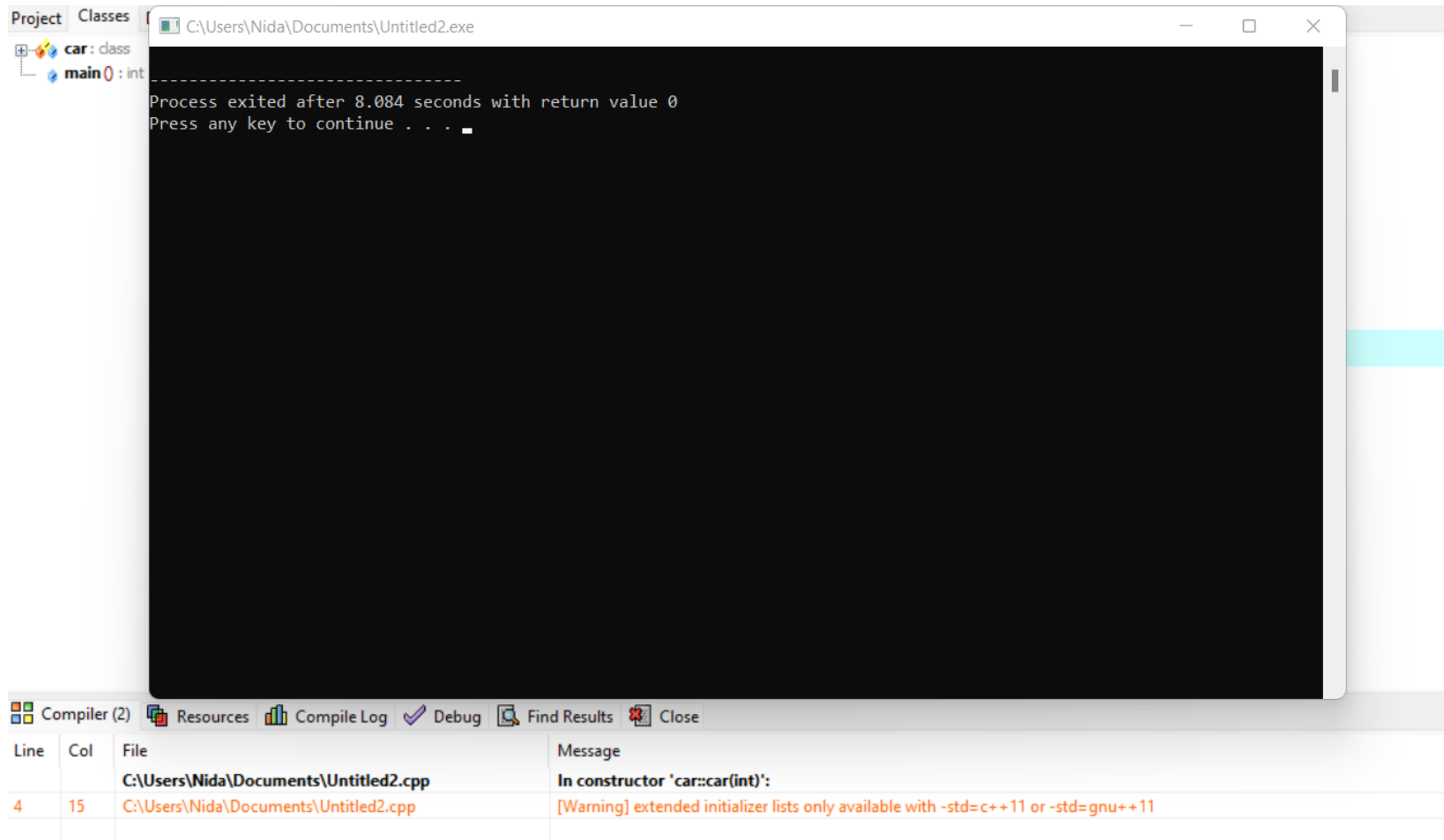
        void print(){
            std::cout<<a;        }

};
int main(){
    car c(10);
    c.print();
}
```

Using {} this is uniform initialization(in C++11) and preferred

```
class car{
    char x;
    public:
        car(int a):x{a}{
        }
};
int main(){
    car c(10);
}
```

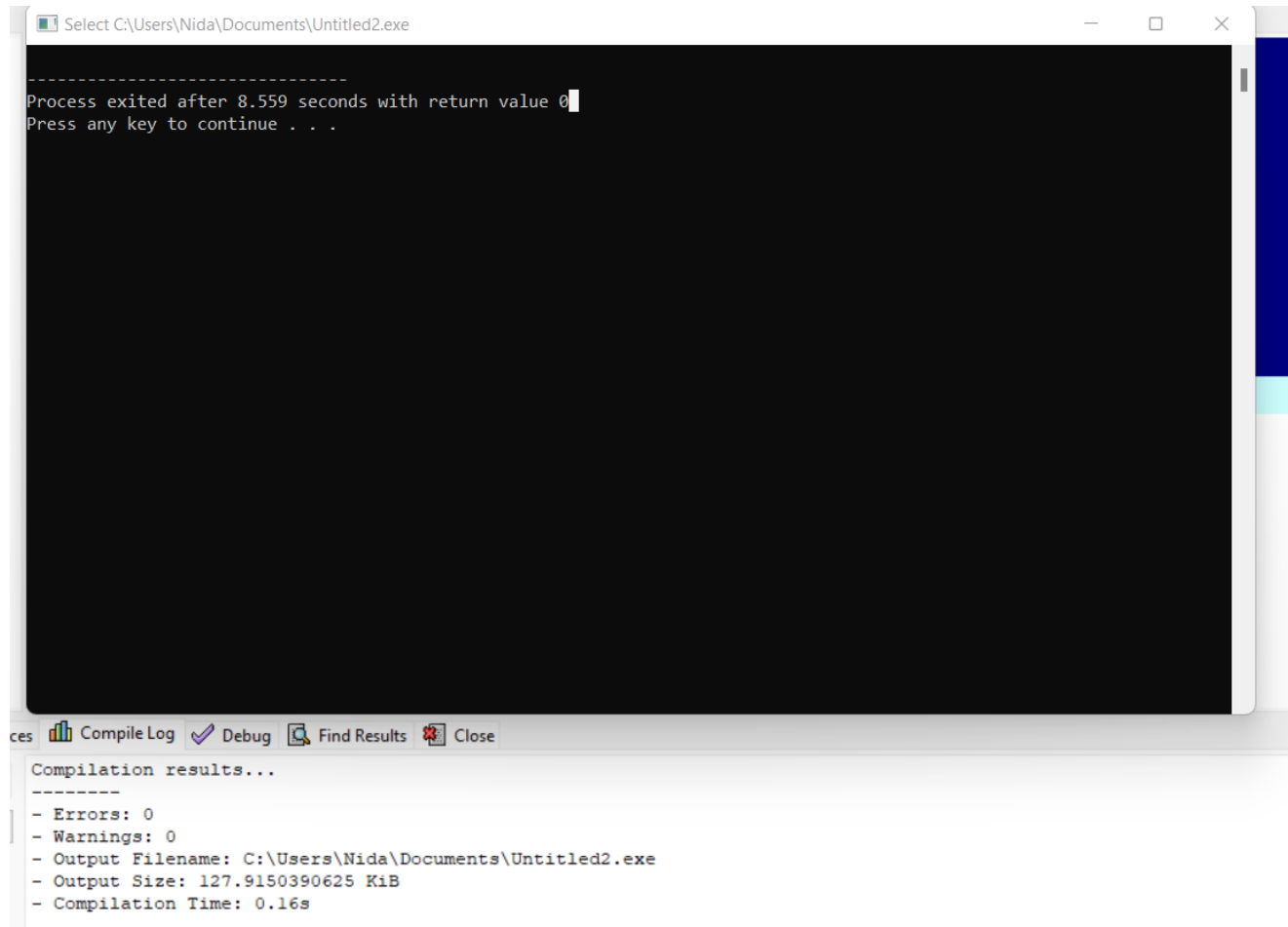
Giving us warning (narrowing conversion)



Using ()

```
class car{  
    char x;  
    public:  
        car(int a):x(a){  
            }  
  
};  
int main(){  
    car c(10);  
}
```

No warning



What Are All Those Places Where_INITIALIZER List Is Must In C++?

1. For Performance reasons:

It is better to initialize all class variables in_INITIALIZER List instead of assigning values inside body.

2. For initialization of non-static const data members(we'll see this in later example)

3. For initialization of base class members

: the parameterized constructor of the base class can only be called using_INITIALIZER List.(don't worry we'll use this concept in inheritance)

4. When constructor's parameter name is same as data member

```
#include <iostream>
class car{
    int a;
    public:
        car(int a):a(a){
        }
        void print(){
            std::cout<<a;
        }
};
int main(){
    car c(10);
    c.print();
}
```

this Pointer is Not Available for Initialization

```
#include <iostream>
class car{
    int a;
    public:
        car(int a):this->a(a){
        }
        void print(){
            std::cout<<a;
        }
};
int main(){
    car c(10);
    c.print();
}
```

this Pointer is Not Available for Initialization

- this is a pointer to the current instance of the object.
- However, at the time of the initializer list execution,
- the object is not fully constructed yet.
- Since this refers to an object that is still being built,
- C++ does not allow you to use this in an initializer list.

5. For initialization of reference members:

```
#include <iostream>
class car{
    int &a;
    public:
        car(int a):a(a){
        }
        void print(){
            std::cout<<a;
        }
};
int main(){
    car c(10);
    c.print();
}
```

Output?

```
#include <iostream>
class car{
    int &a;
    public:
        car(int a){
            this -> a = a;
        }
        void print(){
            std::cout<<a;
        }
};
int main(){
    car c(10);
    c.print();
}
```

output

The screenshot shows a C++ IDE with a code editor and a compiler error message. The code defines a `car` class with a constructor and a `print` method, and a `main` function that creates a `car` object and calls `print`. The constructor is highlighted in red, and the compiler error message at the bottom indicates an uninitialized reference member in the constructor.

```
1 #include <iostream>
2 class car{
3     int &a;
4     public:
5     car(int a){
6         this -> a = a;
7     }
8     void print(){
9         std::cout<<a;
10    }
11 };
12
13 int main(){
14     car c(10);
15     c.print();
16 }
17
```

Compiler (3) Resources Compile Log Debug Find Results Close

Line	Col	File	Message
		C:\Users\Nida\Documents\Untitled2.cpp	In constructor 'car::car(int)':
5	3	C:\Users\Nida\Documents\Untitled2.cpp	[Error] uninitialized reference member in 'int&' [-fpermissive]
3	7	C:\Users\Nida\Documents\Untitled2.cpp	[Note] 'int& car::a' should be initialized

Constant Variables

- The keyword ***const*** be used to declare constant variables
- They must be initialized when they are declared and cannot be modified later
- Using constant variables to specify array size makes program more *scalable*
- Constant variables are also called ***named constants*** or ***read-only variables***

Constant Variables

```
int main()
{
    const int a = 5;
    const int b;      // will cause error
    b = 10;           // will cause error

    const int arr[] = {1, 2, 3, 4, 5};
    arr[0] = 10;      // will cause error
}
```

Member Initialization List

- Constant class members can only be initialized through constructor's member initialization list
- The list of members to be initialized is indicated with constructor as a comma-separated list followed by a colon

Member Initialization List

```
class A{  
    const int x;  
    const int y;  
    public:  
    A ( int val1 , int val2 )  {  
        x=val1; // error  
        x=val2;} // error  
};  
int main() {  
    A a ( 5 , 10 );}
```

Member Initialization List

```
class A{  
    const int x;  
    const int y;  
    public:  
    A ( int val1 , int val2 ) : x ( val1 ) , y ( val2 )  
    {}  
  
};  
  
int main() {  
    A a ( 5 , 10 );}
```

Constant Parameters

```
int func(const int a, int b)
{
    a += 10;           //will cause error
    b += 20;
}
```

CONSTANT MEMBER FUNCTIONS

- Constant member function is the function that cannot modify the data members.
- To declare a constant member function, write the `const` keyword after the closing parenthesis of the parameter list. If there is separate declaration and definition, then the `const` keyword is required in both the declaration and the definition.
- Constant member functions are used, so that accidental changes to objects can be avoided. A constant member function can be applied to a non-const object.
- Keyword, `const` can't be used for constructors and destructors because the purpose of a constructor is to initialize data members, so it must change the object. Same goes for destructors.

```
#include<iostream>
using namespace std;
class Demo {
    int val;
    public:
    Demo(int x = 0) {
        val = x;
    }
    int getValue() const {
        val=2; //[Error] assignment of member 'Demo::val' in read-only object
        return val;
    }
};
int main() {
    Demo d1(8);
    cout << "\nThe value using object d1 : " << d1.getValue();
    return 0;
}
```

principle of least privilege

how the principle of least privilege applies to objects?

Some objects need to be modifiable, and some do not. You may use keyword `const` to specify that an object is not modifiable and that any attempt to modify the object should result in a compilation error.

CONSTANT OBJECTS

- As with normal variables we can also make class objects constant so that their value can't change during program execution. Constant objects can only call constant member functions. The reason is that only constant member function will make sure that it will not change value of the object. They are also called as read only objects. To declare constant object just write const keyword before object declaration.

CONSTANT OBJECTS

- class Demo {
- public:
- int val;
- Demo(int x = 0) {
- val = x;}
- int getValue() const {
- return val; }
- int getValue1() {
- return val;}};
- int main() {
- const Demo d(28);
- cout << "The value using constant object d : " << d.getValue();
- cout << "\nThe value using object d non const func : " << d.getValue1();//error
- d.val=10;// error, can't modify const objects
- return 0;
- }

CONSTANT OBJECTS

- A constructor must be a non-const member function but it can still be used to initialize a const object) shows that it calls another non-const member function. Invoking a nonconst member function from the constructor call as part of the initialization of a const object is allowed. The “constness” of a const object is enforced from the time the constructor completes initialization of the object until that object’s destructor is called.

Principle of Least Privilege???

Constant With Pointers

- There are four ways to use **const** with pointers:
 - Non-constant pointers to non-constant data
 - Non-constant pointers to **constant data**
 - **Constant pointers** to non-constant data
 - **Constant pointers** to **constant data**

Non-constant Pointers to Non-constant Data

- The highest access is granted by a **non-constant pointer to non-constant data**
- Data can be modified through pointer, and pointer can be made to point to other data

Non-constant Pointers With Non-constant Data

```
int main()
{
    int a = 10;
    int b = 50;

    int* pA = &a;
    *pA = 20;
    pA = &b;
}
```

Non-constant Pointers to Constant Data

- Pointer can be modified to point to any other data, but the data to which it points cannot be modified through that pointer

const int * p;

Or

int const *p;

Non-constant Pointers to Constant Data

```
int main()
{
    int a = 10;
    int b = 50;

    const int* pA = &a;
    *pA = 20;           // this line will cause error
    pA = &b;
}
```

```
int main()
{
    int a = 10;
    int b = 50;

    const int* pA;
    pA = &a;
    pA = &b;
    a=23;
}
```

Constant Pointers to Non-constant Data

- Always points to the same memory location, but the data at that location can be modified through the pointer

```
int * const p = &val;
```

```
//here pointer is constant, so we need to assign  
value immediately
```

Constant Pointers to Non-constant Data

```
int main()
{
    int a = 10;
    int b = 50;

    int* const pA = &a;
    *pA = 20;
    pA = &b;           // this line will cause error
}
```

Constant Pointers to Constant Data

- Always points to the same memory location, and the data at that location cannot be modified via the pointer

const int * const p = &val;

or

int const * const p = &val;

Constant Pointers to Constant Data

```
int main()
{
    int a = 10;
    int b = 50;

    const int* const pA = &a;
    *pA = 20;           // cannot do this
    pA = &b;             // cannot do this as well
}
```